BE-JV-005

Aula 04 - Streams

Revisão Aula 03
Kahoot

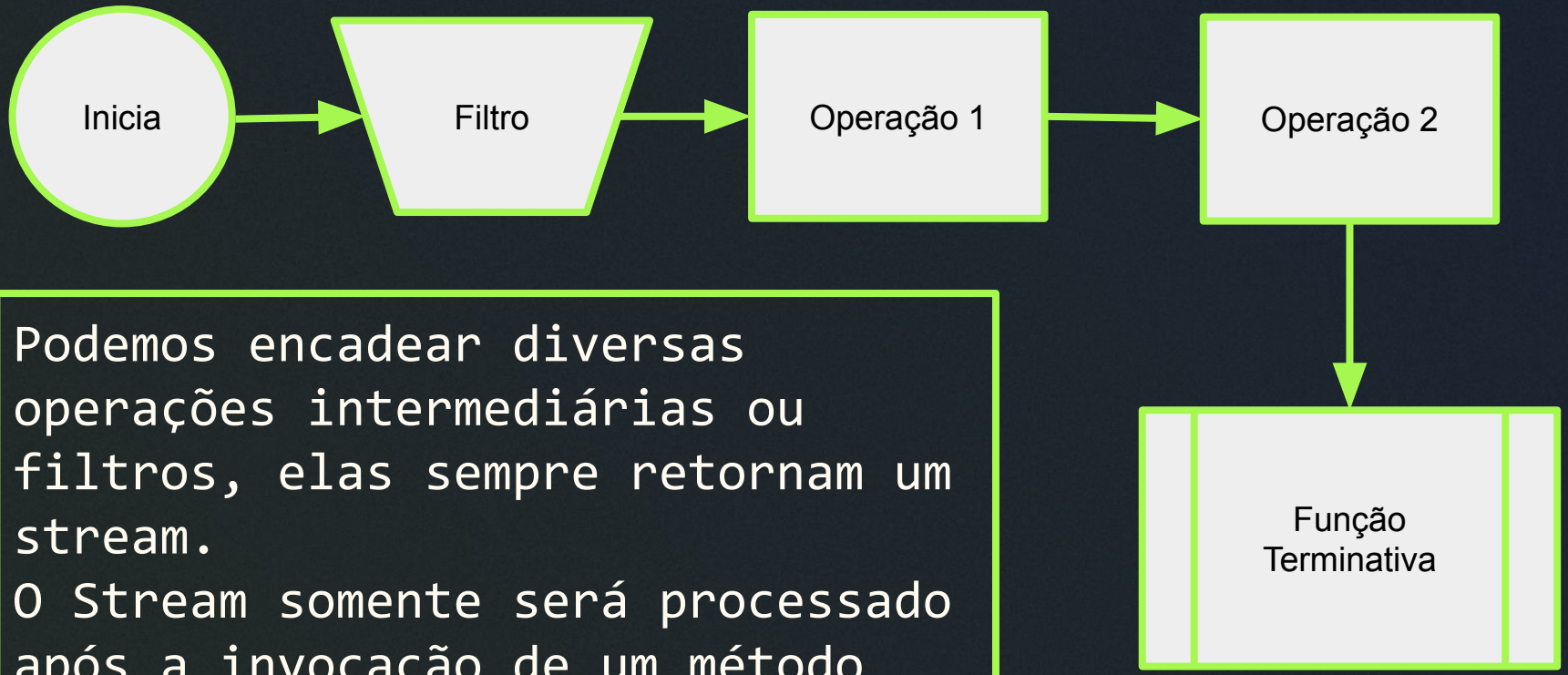
Streams

Fluxo contínuo de dados

Processamento em partes

Necessita ser consumido para processar.
Não pode ser reprocessado após o término.

Streams



Podemos encadear diversas operações intermediárias ou filtros, elas sempre retornam um stream.
O Stream somente será processado após a invocação de um método **terminativo.**

Criando Stream

Método construtor	Uso	Exemplo
Collections.stream()	A forma mais comum. Pega uma coleção já existente (como um List) e a coloca na esteira.	transacoes.stream()...
Stream.of	Cria uma esteira "do zero", passando os elementos soltos separados por vírgula.	Stream.of("A", "B", "C")...
IntStream.range	Cria uma esteira gerando números em sequência (substitui o for clássico de contagem).	IntStream.range(1, 10)...

Operações intermediárias

Método	Uso	Entrada	Exemplo
.filter(...)	Filtra os registros que estão passando pelo stream	Predicate<T>	.filter(t -> t.getValor() > 100)
.map(...)/ .flatMap(...)	Transforma os registros	Function<T, R>	.map(t -> t.getNomeProfissao())
.distinct()	Filtra os registros repetidos	não possui	.distint()
.peek(...)	Dá uma espiadinha sem mexer mexer no stream, utilizado para debug e impressão dos valores durante o processamento	Consumer<T>	.peek(System.out::pri ntln)

Operações terminativas

Método	Uso	Saída	Exemplo
.collect(...)/ .toList()/ .toSet()	Coleta os registros restantes e retorna da forma especificada	List, Set, Map ...	.collect(Collectors.toList())
.count()	Conta a quantidade de registos que sobraram	long	.count()
.reduce(...)	Acumula/Agrega os registros em uma única resposta	Uma única resposta	.reduce(0, (a, b) -> a + b) // retorna a soma
.anyMatch(...)/ .allMatch(...)/ .noneMatch(...)	Faz verificações nos registros. Tem algum X? Todos são X? Nenhum é X?	boolean	.anyMatch(t -> t.getValor() > 10000)
.forEach(...)	Processa os registros	não possui (Void)	.forEach(System.out::println)

Fixação

Exercícios 01

SelecionadorClientesMarketingStream.java

→ Na aula passada, utilizamos o método `removeIf` para modificar uma lista existente. Agora, utilizando a API de **Streams**, receba uma lista de clientes e filtre apenas os que realizaram sua **última compra a mais de 90 dias**, gerando uma nova lista de resultados.

Exemplo:

```
Clientes:
Cliente[nome: Teste1, dataUltimaCompra: 2025-03-05]
Cliente[nome: Teste2, dataUltimaCompra: 2026-03-08]
Cliente[nome: Teste3, dataUltimaCompra: 2025-11-15]
Cliente[nome: Teste4, dataUltimaCompra: 2026-02-22]
Cliente[nome: Teste5, dataUltimaCompra: 2025-10-19]

Clientes selecionados:
Teste1
Teste3
Teste5
```

ada

Exercícios 02

FiltroTransacoes.java

→ Receba uma lista de valores (`BigDecimal`) e utilize Streams para selecionar apenas as transações acima de R$ 500,00, imprimindo cada uma delas no console.

Exemplo:

```
Entrada: [100.00, 600.00, 450.00, 1200.00]
Saída:
600.00
1200.00
```

ada

Exercícios 03

NomesUnicos.java

→ Dada uma lista de nomes de clientes (com repetições e variações de maiúsculas/minúsculas), utilize Streams para padronizar todos os nomes para letras MAIÚSCULAS e garantir que o resultado final não contenha duplicatas.

Exemplo:

```
Entrada: ["Ana", "ana", "Beto", "ANA", "Carlos"]
Saída:
ANA
BETO
CARLOS
```

ada

Exercícios 04

ValidacaoSaldos.java

→ Crie um programa que receba uma lista de saldos de contas (`BigDecimal`) e utilize a API de Streams para realizar as seguintes verificações:

1. Verifique se todas as contas na lista possuem saldo positivo.
2. Verifique se existe pelo menos uma conta na lista com saldo superior a R$ 10.000,00.

Exemplo:

```
Entrada: [100.00, -50.00, 20000.00]
Saída:
Todas positivas? false
Alguma VIP (>10k)? true
```

ada

Exercícios 05

BalancoFinanceiro.java

→ Um banco moderno precisa consolidar o saldo final de seus clientes a partir de uma lista de transações. Implemente um record `Transacao(BigDecimal valor, TipoTransacao tipo)` onde `TipoTransacao` é um `Enum` com os valores `CREDITO` e `DEBITO`. Sua tarefa é processar uma lista dessas transações e produzir o valor final do saldo, considerando que cada tipo de transação afeta o saldo de forma diferente.

Exemplo:

```
Entrada:
Transacao[100.00, CREDITO]
Transacao[50.00, DEBITO]
Transacao[200.00, CREDITO]

Saída: 250.00
```

ada

→ No projeto final, processaremos o arquivo `bank_transactions_data_2.csv`. Como primeiro passo, crie o record `Transaction` conforme especificado (transactionId, accountId, amount, timestamp, type, channel, occupation, loginAttempts, balance) e utilize a API de Streams para transformar as linhas brutas do CSV em objetos desse tipo.

Exemplo:

```
Entrada:
"TX000001,AC00128,14.09,2023-04-11 16:29:14,Debit,San Diego,D000380,162.198.218.92,M015,ATM,70,Doctor,81,1,5112.21,2024-11-04 08:08:08"
"TX000002,AC00455,376.24,2023-06-27 16:44:19,Debit,Houston,D000051,13.149.61.4,M052,ATM,68,Doctor,141,5,13758.91,2024-11-04 08:09:35"
"TX000003,AC00019,126.29,2023-07-10 18:16:08,Debit,Mesa,D000235,215.97.143.157,M009,Online,19,Student,56,1,1122.35,2024-11-04 08:07:04"
"TX000004,AC00070,184.50,2023-05-07 16:32:11,Debit,Raleigh,D000187,200.13.225.150,M002,Online,26,Student,25,1,8569.06,2024-11-04 08:09:06"
"TX000005,AC00411,13.45,2023-10-16 17:51:24,Credit,Atlanta,D000308,65.164.3.100,M091,Online,26,Student,198,1,7429.40,2024-11-04 08:06:39"

Saída:
Transaction[transactionId=TX000001, accountId=AC00128, amount=14.09, timestamp=2023-04-11T16:29:14, type=Debit, channel=ATM, occupation=Doctor,
loginAttempts=1, balance=5112.21]
Transaction[transactionId=TX000002, accountId=AC00455, amount=376.24, timestamp=2023-06-27T16:44:19, type=Debit, channel=ATM, occupation=Doctor,
loginAttempts=5, balance=13758.91]
Transaction[transactionId=TX000003, accountId=AC00019, amount=126.29, timestamp=2023-07-10T18:16:08, type=Debit, channel=Online,
occupation=Student, loginAttempts=1, balance=1122.35]
Transaction[transactionId=TX000004, accountId=AC00070, amount=184.50, timestamp=2023-05-07T16:32:11, type=Debit, channel=Online,
occupation=Student, loginAttempts=1, balance=8569.06]
Transaction[transactionId=TX000005, accountId=AC00411, amount=13.45, timestamp=2023-10-16T17:51:24, type=Credit, channel=Online,
occupation=Student, loginAttempts=1, balance=7429.40]
```

DetectorAlertasAcesso.java

→ Com a lista de objetos `Transaction` mapeada, identifique transações que disparam alertas de segurança baseados no comportamento de acesso: selecione apenas as transações que tiveram mais de 3 tentativas de login.

Exemplo:

Entrada: (As 5 transações produzidas na Parte 1)
Saída:
TX000002

DetectorHorarioSuspeito.java

→ Com a mesma lista de objetos `Transaction`, selecione os IDs das transações ocorridas em horários considerados de risco pelo banco: transações realizadas a partir das 18:00:00 (inclusive).

Exemplo:

Entrada: (As 5 transações produzidas na Parte 1)
Saída:
TX000003

ada

RelatorioFraude.java

→ Consolide os critérios de fraude (Tentativas Excedidas ou Horário Suspeito) em um relatório simplificado. Para cada transação suspeita, gere a saída formatada: `transactionId,timestamp,loginAttempts,amount`.

Exemplo:

Entrada: (As 5 transações produzidas na Parte 1)
Saída:
TX000002,2023-06-27T16:44:19,5,376.24
TX000003,2023-07-10T18:16:08,1,126.29

ada

Projeto final - parte 05 - Ranking de Maiores Transações

TopTransacoesRelatorio.java

→ Um analista financeiro precisa identificar os maiores fluxos de capital. Selecione as 3 transações de maior valor (`amount`) e gere uma saída formatada em ordem decrescente, seguindo o padrão do projeto final.

Exemplo:

```
Entrada: (As 5 transações produzidas na Parte 1)
Saída:
1. ID: TX000002 | Valor: R$ 376,24 | Conta: AC00455 | Data: 27/06/2023
2. ID: TX000004 | Valor: R$ 184,50 | Conta: AC00070 | Data: 07/05/2023
3. ID: TX000003 | Valor: R$ 126,29 | Conta: AC00019 | Data: 10/07/2023
```

Projeto final - parte 06 - Volume Total Movimentado

VolumeTotalRelatorio.java

→ Consolide o valor total movimentado para o relatório do console. Calcule o somatório final líquido (Débitos subtraem e Créditos aumentam o saldo).

Exemplo:

Entrada: (As 5 transações produzidas na Parte 1)
Saída:
Valor total movimentado: R$ -687,97

ada

Obrig.ada